Agentic Orchestration: The Jasper Architecture

A Technical White Paper on Multi-System, Memory-Persistent, Cryptographically-Anchored AI Agent Design

Version: 1.0 — July 2026

Organisation: Jasper / Jasperine AI Systems

Classification: Public Technical Release

Abstract

Jasper is an agentic orchestration platform designed for persistent, multi-system AI-mediated operation in professional and financial environments. This paper presents the architecture of Jasper Version 1.0, comprising eight coordinated subsystems: a typed, importance-weighted memory persistence layer (MemoryBank); a semantic knowledge graph (KnowledgeGraph); an emotional intelligence calibration engine; a proactive monitoring and autonomous task scheduler; a universal financial settlement bridge (URIB) implementing a seven-stage cryptographic pipeline; an intelligent LLM routing layer; a universal application connector framework; and a physical asset tokenisation and GPS-tracking system.

The URIB produces cross-rail settlement across ISO 20022 (pacs.008), Bitcoin (BIP-341 Taproot), XRP Ledger, and wholesale CBDC rails simultaneously, with strict cross-rail value equivalence enforced at Stage 6. Cryptographic accountability is provided through ThreadZero — an append-only, hash-chained audit trail — and a three-layer stack commitment ($C_{stack} = H(h_D \| h_G \| T^*)$) that binds the canonical document hash, semantic graph hash, and truth-chain

anchor into a single verifiable commitment. Settlement roots are further stamped with post-quantum digital signatures (ML-DSA-65, FIPS 204) under JIP-QRM-URIB-01 §6. Identity is managed via decentralised identifiers (DIDs) with rail-specific identity mapping that preserves cross-rail privacy.

This paper is intended for architects, engineers, and technical decision-makers evaluating Jasper for deployment in high-accountability, multi-rail financial environments.

Keywords: agentic orchestration, multi-rail settlement, ISO 20022, Bitcoin Taproot, post-quantum cryptography, memory persistence, knowledge graph, ThreadZero, cross-rail value equivalence

Table of Contents

---

1. Executive Summary

The Jasper Agentic Orchestration Platform addresses a fundamental limitation of current AI systems: the inability to maintain coherent, accountable, and financially capable agency across complex, multi-domain professional environments.

Conventional AI assistants operate statelessly — each interaction begins without prior context, without awareness of ongoing objectives, and without the capacity for autonomous action between user prompts. For professional and institutional deployments — those involving sustained projects, trusted relationships, financial transactions, and regulatory accountability — this stateless model is architecturally insufficient.

Jasper is designed differently. It operates as a persistent, context-grounded agent that maintains memory across sessions, understands the semantic relationships between entities in a user's operational world, monitors for relevant events autonomously, routes computation to the most appropriate model for each task, and executes financial settlement across heterogeneous payment rails — all within a cryptographically auditable framework.

This white paper documents the full technical architecture of Jasper Version 1.0. The platform comprises eight integrated subsystems operating beneath a single orchestration layer:

- **MemoryBank** — A typed, tagged, importance-weighted memory persistence store supporting six memory types, related-memory linking, and a two-tier session/long-term architecture.
- **KnowledgeGraph** — A semantic graph of typed nodes and weighted, typed connections enabling structured reasoning about entities and their relationships.
- **Emotional Intelligence Layer** — A sentiment, energy, and stress-detection engine that calibrates tone and response style to the user's real-time state.
- **Proactive Task Monitor** — A scheduled autonomous task system that monitors defined targets, evaluates trigger conditions, and acts without requiring explicit user prompts.

- **Universal Rail Integration Bridge (URIB)** — A seven-stage settlement pipeline producing simultaneous cross-rail settlement across ISO 20022, Bitcoin BIP-341 Taproot, XRP Ledger, and wholesale CBDC, with strict value equivalence enforced at Stage 6 and a full cryptographic proof chain.
- **Model Router** — An intelligent LLM selection layer spanning GPT-5, Gemini 3, and Claude model families, routing each task to the optimal model based on complexity, type, and latency requirements.
- **Universal App Connector** — An integration framework for Base44 applications enabling cross-app data access and multi-system workflow coordination.
- **Asset Records** — A physical asset management system with GPS tracking, vault storage, document linkage, and Bitcoin-anchored provenance via the Satoshi Covenant.

The cryptographic foundation runs beneath all subsystems. **ThreadZero** provides an append-only, hash-chained audit trail in which each event incorporates all prior events — making any record tampering detectable by any party holding the final truth anchor (T*). The **Stack Commitment** ($C_{stack} = H(h_D \| h_G \| T^*)$) binds the canonical document hash, semantic graph hash, and truth-chain anchor into a single cryptographic commitment. The **Bridge Commitment** (C_{bridge}) binds all rail states together, providing verifiable proof of cross-rail consistency. Settlement commitments are stamped with post-quantum digital signatures (ML-DSA-65) in accordance with JIP-QRM-URIB-01 §6, providing forward security against quantum-capable adversaries.

Jasper does not merely respond to queries. It remembers, reasons, monitors, routes, settles, and proves — across every session, every subsystem, and every rail it is connected to. The result is an agent that functions as a trusted operational partner: persistent in memory, rigorous in accountability, and capable in finance.

Key Capabilities at a Glance:

Capability	Implementation
Persistent memory	Typed, importance-weighted MemoryBank with session and long-

Capability	Implementation
	term tiers
Semantic knowledge	KnowledgeGraph with typed nodes and weighted relational edges
Emotional calibration	Real-time sentiment, energy, and stress detection for adaptive tone
Proactive monitoring	Scheduled autonomous tasks with configurable trigger conditions
Multi-rail settlement	URIB 7-stage pipeline: ISO 20022, BTC Taproot, XRP, CBDC
Cryptographic accountability	ThreadZero hash chain + Stack Commitment + PQ signatures (ML-DSA-65)
Model routing	Dynamic LLM selection across GPT-5, Gemini 3, and Claude families
Physical asset provenance	GPS-tracked asset records anchored to Bitcoin via Satoshi Covenant

---

2. Introduction to Agentic Orchestration

2.1 What Makes a System "Agentic"

A traditional AI assistant responds to a prompt. An agent operates with:

1. **Persistence** — state survives beyond a single interaction
2. **Autonomy** — it can act without being explicitly prompted (proactive monitoring)
3. **Tool use** — it can invoke external systems, not just generate text
4. **Memory** — it recalls prior context and learns from it
5. **Judgement** — it routes, selects, and decides between options

6. **Accountability** — its actions are logged, verifiable, and auditable

Jasper possesses all six properties. The orchestration layer is what coordinates them — deciding when to recall memory, when to route to a different model, when to trigger a proactive task, and when to emit a settlement message.

2.2 The Orchestration Challenge

The core challenge of agentic orchestration is coherence across heterogeneous systems. Jasper integrates:

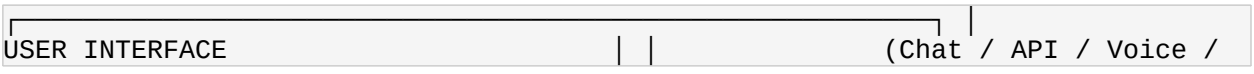
- A memory store (MongoDB-backed document store with typed entities)
- A knowledge graph (node-and-edge structure with typed connections)
- An emotional context engine
- A proactive task scheduler
- A financial settlement bridge with cryptographic proof chain
- An LLM router across multiple model providers
- An application connector framework

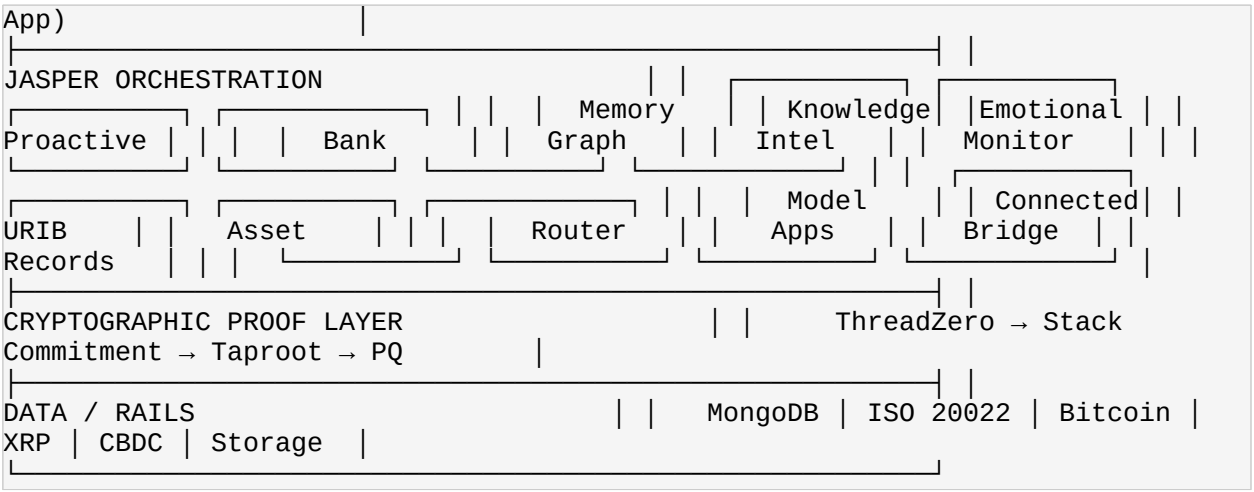
Each of these has its own data model, its own access patterns, and its own failure modes. The orchestration layer's responsibility is to ensure they compose into a coherent agent experience — one where the user perceives a single, intelligent operational partner rather than a collection of disconnected tools.

The design principle that holds this together: every action is contextually grounded. Memory provides the context. The knowledge graph provides the structure. Emotional intelligence provides the calibration. The proof chain provides the accountability.

---

3. The Jasper Stack — Architectural Overview





The stack is layered. The user interacts only with the top layer. The orchestration layer coordinates eight subsystems. The cryptographic proof layer runs beneath everything, providing an audit trail for every significant action. The data/rails layer represents the external systems Jasper can read from and write to.

3.1 Design Principles

7. **Context first** — every response is grounded in recalled memory and knowledge graph state
8. **Typed everything** — memories have types, nodes have types, tasks have types, emotions have types. This enables precise retrieval and reasoning.
9. **Proof by default** — significant actions (especially financial) produce cryptographic proofs that can be independently verified
10. **Graceful degradation** — if the URIB cannot reach a rail, the audit trail still records what happened. If a proactive task fails, it retries. If a memory is expired, it is simply not recalled.
11. **Human in the loop** — Jasper recommends, suggests, and prepares, but the user is the decision-maker for all consequential actions.

---

4. Memory Systems (MemoryBank)

4.1 Overview

The MemoryBank is Jasper's persistent memory store. It allows the agent to remember facts, preferences, goals, relationships, project details, and conversation highlights across sessions. Unlike a simple key-value store, MemoryBank entries are typed, tagged, importance-weighted, and contextually enriched.

4.2 Memory Types

Type	Purpose	Example
`fact`	Objective information	"User's company is registered in Delaware"
`preference`	User preferences	"User prefers direct, no-nonsense tone"
`goal`	Objectives to track	"User wants to complete the white paper by August"
`relationship`	People and connections	"Sarah is the user's CFO"
`project`	Ongoing work	"Building a cross-rail settlement prototype"
`conversation_highlight`	Key moments	"User expressed strong interest in ISO 20022 compliance"

4.3 Memory Schema

```
{   "memory_type": "fact",   "content": "The actual memory content string",
"tags": ["finance", "compliance"],   "importance": 8,   "context":
{     "source": "conversation",     "conversation_id": "abc-123",
"confidence": 0.9   },   "related_memories": ["mem_id_1", "mem_id_2"],
"access_count": 3,   "last_accessed": 1789234567,   "expires_at": null }
```

4.4 Importance Weighting

Each memory carries an importance score from 0–10. This is used for:

- **Prioritisation** during recall — higher-importance memories are surfaced first
- **Retention decisions** — low-importance, low-access memories are candidates for expiry
- **Context budgeting** — when assembling a prompt, importance determines what fits within the context window

4.5 Related Memories & Graph Linking

Memories can reference other memories via `related_memories` IDs. This creates a secondary graph structure — alongside the formal KnowledgeGraph — that captures associative links between pieces of information the agent has encountered. When a memory is recalled, related memories can be fetched to provide richer context.

4.6 Expiration & Lifecycle

Memories can carry an optional `expires_at` timestamp. This is useful for time-bound information (e.g., "User is travelling to London next week" does not need to persist indefinitely). Expired memories are excluded from recall but can be explicitly queried for audit purposes.

4.7 Access Tracking

Every memory records `access_count` and `last_accessed`. This provides:

- A signal for what the agent is actively using versus what is going stale
- The ability to implement frequency-based recall (recently accessed memories are likely relevant)
- An audit trail of what information has been considered

4.8 Two-Tier Memory: Session vs. Long-Term

Jasper operates a two-tier memory architecture:

12. **Session memory** (via the `save_memory` / `delete_memory` tool layer) — scoped to `global` (shared across users) or `user` (specific to the current user). These are lightweight facts and preferences.
13. **Structured MemoryBank** — the full-featured store with types, tags, importance scoring, relational linking, and expiration management. Used for richer, more structured persistent information.

This two-tier design allows Jasper to quickly persist a simple preference while also maintaining structured project knowledge with full metadata.

---

5. Knowledge Graph (KnowledgeNode)

5.1 Overview

While the MemoryBank stores information as discrete memories, the KnowledgeGraph stores structured knowledge as typed nodes with weighted, typed connections. This is Jasper's semantic understanding layer — it does not merely know facts; it knows how facts relate to each other.

5.2 Node Types

Type	Purpose
`person`	People — colleagues, contacts, family members
`project`	Ongoing work and initiatives
`concept`	Ideas, topics, domains
`location`	Physical or virtual places
`organization`	Companies, institutions
`event`	Scheduled or historical occurrences

Type	Purpose
`document`	Files, papers, contracts

5.3 Node Schema

```
{   "node_type": "person",   "name": "Sarah Chen",   "description": "CFO at the user's company",   "properties": {     "title": "Chief Financial Officer",     "email": "sarah@company.com",     "role": "financial_approver"   },   "connections": [     {       "target_node_id": "node_project_settlement",       "relationship": "manages",       "strength": 0.9     },     {       "target_node_id": "node_org_company",       "relationship": "works_at",       "strength": 1.0     }   ],   "metadata": {     "source": "conversation",     "confidence": 0.95,     "created_date": "2026-07-19"   } }
```

5.4 Connections: Relationship Types & Strength

Each connection carries:

- A `target_node_id` — the node it connects to
- A `relationship` — a string describing the nature of the connection (e.g., "manages", "works_at", "references", "depends_on", "owns")
- A `strength` — a numeric weight (0.0–1.0) indicating the strength or importance of the relationship

This allows the graph to represent not just that two things are connected, but how strongly and in what way.

5.5 Querying the Graph

The KnowledgeGraph supports MongoDB-style query syntax, enabling:

- **Type filtering** — retrieve all nodes of a given type
- **Property matching** — find nodes by specific property values
- **Connection traversal** — start from one node and follow its connections
- **Sorting and limiting** — for ranked, bounded retrieval

5.6 Graph vs. Memory: When to Use Which

- **MemoryBank** is for narrative information — things that occurred, things that were said, preferences expressed. It is associative and temporal.
- **KnowledgeGraph** is for structured information — entities and their relationships. It is relational and spatial.

The two systems are complementary. A memory might record: "Sarah mentioned the Q3 audit is a concern." A knowledge node would record: "Sarah is the CFO, connected to the company node, which is connected to the Q3 audit project node."

---

6. Emotional Intelligence Layer

6.1 Overview

The EmotionalContext system allows the agent to detect and respond to the user's emotional state — not as a stylistic feature, but as a practical calibration mechanism for communication effectiveness.

6.2 Emotional Context Schema

```
{   "sentiment": "positive",   "emotions_detected": ["excited", "curious"],
"energy_level": 7,   "stress_indicators": [],   "recommended_tone": "casual",
"confidence": 0.82,   "conversation_context": {     "topic": "white_paper",
"session_id": "abc-123"   } }
```

6.3 Sentiment Classification

Level	Description
`very_positive`	Enthusiastic, delighted, energised
`positive`	Content, interested, optimistic
`neutral`	Factual, calm, business-as-usual
`negative`	Frustrated, concerned, worried

Level	Description
`very_negative`	Distressed, angry, overwhelmed

6.4 Emotion Detection

Beyond overall sentiment, specific emotions are detected: `excited`, `frustrated`, `confused`, `curious`, `anxious`, `confident`, `overwhelmed`, and others. These are used to fine-tune the response — a confused user requires clearer explanations; a frustrated user requires efficiency and directness.

6.5 Energy Level

A 0–10 scale representing the user's apparent energy and engagement. This influences:

- **Response length** — low energy produces shorter, more direct responses
- **Suggestion density** — low energy produces fewer options and more decisive recommendations
- **Tone** — high energy permits matching enthusiasm; low energy calls for calm and support

6.6 Stress Indicators

The system flags stress indicators — patterns in language that suggest the user is under pressure. When stress is detected, the recommended tone shifts to `empathetic` or `direct`, reducing complexity rather than compounding it.

6.7 Recommended Tone

Tone	When Used
`professional`	Business context, formal communication
`casual`	Default — warm, conversational

Tone	When Used
`playful`	Light moments, creative work
`serious`	Important decisions, critical information
`empathetic`	Stress detected, difficult topics
`direct`	Time-pressured, frustrated user, urgent matters

6.8 Confidence

The emotional assessment carries a confidence score (0.0–1.0). Low confidence causes the system to default more strongly to the casual/professional baseline. High confidence permits more aggressive tone adaptation.

---

7. Proactive Monitoring & Autonomous Tasks

7.1 Overview

The ProactiveTask system allows the agent to monitor for conditions and take action when they are met — whether alerting the user to a deadline, summarising news on a tracked topic, or suggesting an action based on observed patterns. Jasper does not require prompting to initiate these actions.

7.2 Task Schema

ProactiveTasks are managed in a centralised registry, each represented as a structured document with a defined type drawn from five categories: `monitor`, `alert`, `brief`, `reminder`, and `suggestion`. Each task is configured along three core axes. The first is **trigger conditions** — rule-based predicates that the system evaluates at each frequency cycle to determine whether action is warranted. The second is **execution frequency** — the polling cadence, ranging from continuous real-time

monitoring for fast-moving targets to weekly evaluation for strategic, low-frequency reviews. The third is **priority level** — the escalation logic that governs how and when a triggered result is surfaced to the user, from a low-priority suggestion offered at a natural conversational break to an urgent alert raised immediately ahead of all other output. The schema below gives the complete structure of a ProactiveTask document.

```
{   "task_name": "Monitor Q3 Audit Deadline",   "task_type": "alert",
"monitoring_target": {     "type": "deadline",     "date": "2026-09-30",
"project": "Q3_audit"   },   "trigger_conditions": {     "days_before": 7,
"status": "incomplete"   },   "frequency": "daily",   "priority": "high",
"is_active": true,   "next_check": 1789234567,   "action": {     "type":
"notify_user",     "message": "Q3 audit deadline is 7 days away. Current
status: incomplete."   } }
```

7.3 Task Types

Type	Description
`monitor`	Passively watch a target and log observations
`alert`	Notify the user when trigger conditions are met
`brief`	Produce a summary or briefing on a topic at scheduled intervals
`reminder`	Time-based reminders for tasks or events
`suggestion`	Proactively suggest actions based on observed patterns

7.4 Frequency Options

- `realtime` — continuous monitoring (for fast-moving targets such as market prices)
- `hourly` — evaluated every hour
- `daily` — evaluated once per day
- `weekly` — evaluated once per week

7.5 Priority Levels

`low` → `medium` → `high` → `urgent`. Priority determines how aggressively the system surfaces the result. An urgent alert is raised immediately; a low-priority suggestion waits for a natural break in conversation.

7.6 Monitoring Targets

The `monitoring_target` field is flexible and can represent:

- **Market prices** — threshold crossings and volatility events
- **Deadlines** — project milestones, regulatory filings
- **News topics** — keyword or entity tracking
- **System states** — connected application statuses, asset locations
- **Patterns** — behavioural patterns the agent has learned to watch for

7.7 Trigger Conditions

Triggers are rule-based conditions evaluated against the monitoring target. Examples:

- Price crosses a defined threshold
- Deadline is within N days and project status is incomplete
- News mentions exceed a volume threshold
- GPS location changes beyond a defined geofence

7.8 Actions

When triggered, a task can:

- Notify the user with a message
- Create a memory entry
- Create or update a knowledge node
- Suggest a follow-up action

- Log an audit entry

---

8. The Universal Rail Integration Bridge (URIB) — ISO 20022 & Cross-Rail Settlement

8.1 Overview

The **Universal Rail Integration Bridge (URIB)** is Jasper's financial settlement engine. It accepts a raw transaction document as input and executes a seven-stage deterministic pipeline that produces five simultaneous outputs: an ISO 20022 pacs.008 credit transfer message, a Bitcoin BIP-341 Taproot commitment transaction, an XRP Ledger payment instruction, a wholesale CBDC proof, and a full cryptographic audit trail. The pipeline is deterministic in the strict sense — given the same input document, it will produce identical outputs on every execution, and every output is reproducible by any party who holds the input and the pipeline specification.

The architectural significance of producing all five outputs simultaneously from a single input is not that they are generated in parallel — it is that they constitute a single, coherent settlement event. Every output is cryptographically bound to the same canonical document hash, the same semantic graph hash, and the same truth-chain anchor. There is no version of the ISO 20022 message that is not also committed to the same Bitcoin Taproot key, and no version of the XRP payment instruction that is not also anchored to the same ThreadZero truth chain. The five outputs are not translations of the same transaction; they are five expressions of one irreversible, cryptographically unified settlement act.

All outputs carry enforced cross-rail value equivalence — the same notional value, expressed in each rail's native format, committed to the same cryptographic anchor.

8.2 The 7-Stage Pipeline

```
Stage 1: Canonical Tokenization     → h_D (document hash) Stage 2: Semantic
Tokenization       → h_G (semantic graph hash) Stage 3: ThreadZero Truth
Chain      → T* (truth anchor) Stage 4: Stack Commitment            → C_stack
Stage 5: Bitcoin/Taproot Layer       → P' (taproot key) Stage 6: URIB Rail
Mapping           → S_r (rail states) Stage 7: Settlement Emission         →
ISO 20022 / BTC / XRP / CBDC
```

8.3 Stage 1: Canonical Tokenization

Purpose: Produce a deterministic, order-stable representation of the transaction document.

The raw document (e.g., `{ debtor: "Alice", creditor: "Bob", amount: 1000, currency: "USD" }`) is processed as follows:

The raw document fields are first sorted deterministically — alphabetically by field name — so that the representation is stable regardless of the order in which fields were supplied. Each field is then assigned a type token (string, number, array, or equivalent) to fix the interpretation of every value against the schema. The document is assigned the schema identifier `iso20022_urib_v1`, binding the canonical representation to a specific version of the pipeline specification. The typed, sorted document is then serialised to canonical JSON — sorted keys, no whitespace, no optional formatting — producing a byte string whose content is entirely determined by the input and the schema, with no representational ambiguity. Finally, the serialised byte string is hashed with SHA-256 to produce the output of this stage. The output is `h_D` — the canonical document hash — which commits the transaction document to a fixed, order-stable representation that is reproducible by any party given the same input document and the same schema version.

Output: `h_D` — the canonical document hash.

```
D = { f₁, f₂, …, fₙ } canon_serialize(D) → B h_D = H(B)
```

8.4 Stage 2: Semantic Tokenization

Purpose: Build a semantic graph representing the obligations and rights in the transaction.

The document fields are mapped to a typed graph:

The debtor and creditor in the transaction document become typed **party nodes** in the semantic graph — each carrying its identity, role, and rail-specific addressing. The transaction itself — its amount, currency, and settlement terms — becomes a **contract node** that represents the agreement as a structured object rather than a collection of fields. The payment obligation flowing from debtor to creditor becomes an **obligation node** that gives the obligation independent existence in the graph, separable from both the parties and the contract. The three elements are then connected by typed directional **edges** — `owes` from the debtor party node to the obligation node, `right` from the obligation node to the creditor party node, and `references` from the obligation node to the contract node — producing a structured semantic graph that represents the legal and financial meaning of the transaction, not merely its raw data fields.

Invariants enforced:

Before the pipeline can advance to Stage 3, the semantic graph must satisfy two invariants. The first is **value conservation**: the sum of all debt obligations represented in the graph must equal the sum of all credit rights, ensuring that the obligation is perfectly balanced and that no value is created or destroyed by the semantic layer. The second is **duality**: every obligation node of the form obligation(A→B) must have a corresponding right node of the form right(B←A) — every debt has a legally corresponding credit entitlement, and the graph is structurally rejected if this symmetry does not hold. A graph that fails either invariant does not proceed to Stage 3; the pipeline terminates with a semantic validation error, and no settlement messages are produced.

Output: `h_G` — the semantic graph hash.

8.5 Stage 3: ThreadZero Truth Chain

Purpose: Create an append-only, hash-chained audit trail of every event in the pipeline.

Each event is structured as:

```
{   "timestamp": 1789234567,   "actor": "jasper",   "action": "TOKENIZE",
"payload": { "h_doc": "...", "h_sem": "..." } }
```

The chain is constructed as:

```
T₀ = H(E₀)                    // First event T(i+1) = H(Tᵢ ‖ E(i+1))     //
Each subsequent event T* = Tₙ                        // Final truth anchor
```

Each anchor incorporates all prior events. Tampering with any event changes all subsequent anchors. Any party holding a copy of T* can detect tampering.

Events recorded in a typical settlement:

Six events are recorded in a typical settlement pipeline, appended to the chain in strict execution order. `TOKENIZE` records the computation of both the canonical document hash `h_D` and the semantic graph hash `h_G`, fixing the pipeline's input commitment at the earliest possible stage. `SEMANTIC_COMMIT` records the finalisation of the typed graph structure — the point at which the obligation and right nodes are established and their relationships sealed. `INVARIANT_PASS` records that both semantic invariants — value conservation and duality — were evaluated and satisfied, constituting a verifiable proof that the pipeline was not permitted to advance on a malformed or unbalanced transaction. `BTC_TAPROOT_COMMIT` records the computation of the Taproot output key P', anchoring the stack commitment to the Bitcoin blockchain layer. `URIB_BRIDGE_COMMIT` records the computation of the cross-rail bridge commitment C_bridge, binding all four rail states into a single verifiable hash. `SETTLEMENT_EMIT` records the emission of the final settlement messages to all four rails, marking the point at which the settlement became irrevocable. The combined hash chain of these six events produces the truth anchor T* that is carried forward into Stage 4 as the processing history commitment.

8.6 Stage 4: Stack Commitment

Purpose: Bind the three layers (canonical, semantic, truth chain) into a single commitment.

```
C_stack = H(h_D ‖ h_G ‖ T*)
```

The stack commitment is a single hash that cryptographically binds the document, its semantic meaning, and its complete processing history. Any change to any of the three layers invalidates C_stack.

8.7 Stage 5: Bitcoin/Taproot Layer

Purpose: Anchor the commitment to the Bitcoin blockchain using BIP-341 Taproot.

The internal public key P_int is tweaked using the stack commitment:

```
P' = P_int + H_tag("TapTweak", P_int ‖ C_stack) · G
```

This produces a Taproot output key P' that commits to C_stack without revealing it on-chain. The commitment is embedded in the tweak, not in a script.

Satoshi binding: A specific satoshi (ordinal index 0) within the UTXO is bound to the commitment:

```
{   "ordinal_index": 0,   "utxo_ref": "utxo:genesis:0",   "h_doc": "...",
"h_sem": "...",   "t_anchor": "...",   "c_stack": "..." }
```

8.8 Stage 6: URIB Rail Mapping

Purpose: Project the settlement onto each target rail while maintaining value equivalence.

For each rail r ∈ {BTC, XRP, ISO, CBDC}:

For each rail r ∈ {BTC, XRP, ISO, CBDC}, two operations are performed in sequence. First, a **rail-specific state object** S_r is computed, containing the settlement value expressed in the rail's native denomination and format, the rail identity of the transacting parties, and the stack commitment C_stack as the binding proof that this rail state was produced from the same canonical document and semantic graph as every other rail state in the set. Second, an **identity mapping** Ψ_r(DID) is applied to translate the user's decentralised identifier into the address format required by that rail — the mapping is described in the table below, and identifiers are derived via hashing rather than directly from each other, so that a party who knows one rail identity cannot derive any other rail identity from it, preserving cross-rail privacy.

Identity mapping examples:

Rail	Identifier Format
ISO 20022	`IBAN_<hash>`
Bitcoin	`bc1p<hash>`
XRP	`r<hash>`
CBDC	`CBDC_<hash>`

Cross-rail invariant: `val(S_BTC) = val(S_XRP) = val(S_ISO) = val(S_CBDC)` — the value must be identical across all rails. If it is not, the pipeline fails with a cross-rail invariant violation.

Bridge commitment: `C_bridge = H(canonical(sort(rail_states)))` — a single hash binding all rail states together.

8.9 Stage 7: Settlement Emission

Purpose: Emit the actual settlement messages for each rail.

ISO 20022 pacs.008:

```
{   "message_type": "pacs.008.001.10",   "group_header": {     "msg_id":
"URIB<timestamp>",     "cre_dt_tm": "2026-07-19T23:44:53Z",     "nb_of_txs":
"1",     "sttlm_inf": { "sttlm_mtd": "CLRG" },     "urib_stack_commitment":
"<C_stack>"   },   "credit_transfer_tx": {     "pmt_id": { "end_to_end_id":
"<message_id>" },     "instd_amt": { "amount": 1000, "ccy": "USD" },
"cdtr_agt": { "fin_instn_id": { "bicfi": "BIC_CREDITOR" } },     "dbtr_agt":
{ "fin_instn_id": { "bicfi": "BIC_DEBTOR" } },     "thread_anchor":
"<T*>"   } }
```

Bitcoin: Taproot transaction with the commitment embedded in the output key.

XRP: Payment transaction with the stack commitment in the memo field and a destination tag derived from the thread anchor.

CBDC: Wholesale CBDC proof with the stack commitment as the proof hash.

8.10 The Full Orchestration Output

When the `orchestrate` action completes, the URIB returns a structured response document that contains the complete cryptographic and settlement record of the transaction — every hash, every commitment, every rail state, and every emitted message, assembled into a single verifiable object.

The response contains eleven fields. `h_doc` and `h_sem` are the canonical document hash and semantic graph hash produced in Stages 1 and 2, fixing the input commitment of the pipeline. The `thread_anchor` T* and its accompanying event count represent the complete ThreadZero truth chain produced during pipeline execution. `c_stack` is the Stage 4 stack commitment that binds all three prior layers — document, semantics, and processing history — into a single hash. The `taproot_key` P' and satoshi bindings are the Bitcoin-layer outputs produced by the BIP-341 Taproot operation in Stage 5. `rail_states` contains the per-rail settlement state objects for all four rails, each carrying the settlement value in its rail-native denomination. `rail_identities` contains the rail-specific identity mappings produced by the Ψ_r(DID) function at Stage 6. `c_bridge` is the bridge commitment

binding all four rail states into a single verifiable hash. `iso20022_messages` contains the emitted pacs.008 credit transfer messages. `btc_tx` and `xrp_tx` contain the Bitcoin and XRP transaction structures respectively. `audit` contains the full ThreadZero audit trail. The complete presence of all eleven fields in a valid response constitutes cryptographic proof that the settlement was executed in full, in order, and in compliance with the cross-rail value equivalence invariant.

---

9. ThreadZero — The Cryptographic Truth Chain

9.1 Overview

ThreadZero is Jasper's native accountability layer. It is not a logging subsystem added to the agent as an afterthought — it is woven into the execution path of every significant action the system takes, operating as the mechanism through which the agent's own processing becomes externally verifiable. Every financial settlement, every memory write of consequence, and every proactive action the agent takes autonomously passes through ThreadZero before it is considered complete. The audit record ThreadZero produces is not a summary of what happened — it is a cryptographically ordered transcript of what happened, produced as the events occurred, in the sequence they occurred, with each event's position in the sequence mathematically committed.

Two properties make ThreadZero more than an audit log. The first is that it is **append-only**: events can be added to the chain but can never be removed or altered without detection. There is no administrative pathway to remove an event from the chain once it has been committed — the data structure does not support deletion, and any system that claims to present a valid ThreadZero chain with an event missing from it presents a chain whose verification will fail. The second property is that the chain is **hash-chained**: each event anchor incorporates all prior anchors in its computation, so any retroactive modification to any event in the sequence invalidates every anchor that follows it. To falsify event E_3 in a ten-event chain, an adversary must recompute T_3 through T_{10} — and the final anchor T* will

not match the T* that was published at settlement time. Any party who holds a copy of the published T* can detect the tampering without any access to Jasper's internal state.

9.2 Hash Chain Structure

```
T₀ = H(E₀) T(i+1) = H(Tᵢ ‖ E(i+1)) T* = Tₙ
```

The chain is constructed from three elements. E_i is the i-th **event** — a structured record containing a Unix timestamp marking the precise moment the event occurred, a DID-format actor identifier identifying the system component that produced the event, an action label drawn from the pipeline's defined event vocabulary, and a payload containing the cryptographic outputs of that action (hashes, commitments, and transaction identifiers as appropriate for the event type). T_i is the i-th **truth anchor** — the SHA-256 hash of the concatenation of the prior anchor T_{i-1} and the current event E_i, binding the current event to all events that preceded it. T* is the **final truth anchor** — the truth anchor produced at the conclusion of the chain, representing the cumulative, order-committed digest of every event that occurred during the pipeline execution. It is T* that is carried into the stack commitment at Stage 4, and T* that is published in the settlement response as the verifiable proof of the complete processing history.

Each anchor incorporates all prior events. To tamper with event E_3, an adversary must recompute T_3, which changes T_4, which changes T_5 — all the way to T*. Any party holding a copy of T* can detect the tampering.

9.3 Event Structure

```
{   "timestamp": 1789234567890,   "actor": "did:jasper:user:abc123",
"action": "BTC_TAPROOT_COMMIT",   "payload": {     "p_prime":
"taproot_a1b2c3d4_e5f6g7h8",     "bindings": 1   } }
```

9.4 Audit Trail

The full audit record produced by ThreadZero at the conclusion of a settlement pipeline contains six fields. The `truth_anchor` T* is the final hash anchor representing the entire event sequence in a single verifiable digest — it is the primary artefact of verification and the field against which any external challenge to the settlement record is resolved. The `event_count` records the total number of

events appended to the chain during the pipeline execution, providing a first-pass integrity signal: a chain with fewer events than expected indicates that a pipeline stage did not complete. The `stack_commitment` C_stack is the Stage 4 commitment that binds the canonical document hash, semantic graph hash, and truth anchor together, allowing the audit record to serve simultaneously as a settlement receipt and a document integrity proof. The `bridge_commitment` C_bridge is the Stage 6 commitment binding all four rail states, providing proof that the cross-rail settlement was executed as a single, mutually consistent event. The complete event array follows — each entry carrying a Unix timestamp, the DID-format actor identifier of the producing component, the action label, and the cryptographic payload — so that the full processing history is present in the audit record and can be replayed for independent verification. Finally, `generated_at` records the ISO 8601 timestamp at which the audit record was produced, providing a wall-clock reference for the settlement event.

This audit trail is stored in an AuditLog entity, providing a persistent, queryable record of every settlement the URIB processes.

9.5 Verification

The `verify` action allows any party to recompute the stack commitment from the component hashes:

```
Input: h_doc, h_sem, t_anchor, c_stack Recompute: C_stack = H(h_doc ‖ h_sem ‖ t_anchor) Result: valid (if recomputed == provided) or invalid
```

This enables independent verification without trusting Jasper — only the three component hashes and the claimed commitment are required.

---

10. Stack Commitment & Bitcoin/Taproot Anchoring

10.1 The Stack Commitment

The stack commitment C_stack = H(h_D ‖ h_G ‖ T*) is the cornerstone of Jasper's financial integrity. It binds three independent layers:

C_stack binds three independent layers, each of which answers a distinct evidential question about the settlement. The **document hash** h_D answers the question of what the transaction was — it commits the raw transaction document to a canonical, reproducible representation that cannot be altered without invalidating the commitment, providing proof of the transaction's content at the moment it entered the pipeline. The **semantic graph hash** h_G answers the question of what the transaction means — it commits the typed graph of obligations and rights that the pipeline attributed to the document, ensuring that the legal meaning assigned to the transaction is fixed alongside the transaction itself and cannot be subsequently reinterpreted. The **truth anchor** T* answers the question of how the transaction was processed — it commits the complete, ordered sequence of every pipeline stage that the settlement passed through, in the order those stages executed, ensuring that the processing history is as immutable as the transaction content. Together, the three layers ensure that a settlement cannot be disputed on any of its three most fundamental dimensions: what it was, what it meant, and how it was handled. Any dispute about a settled transaction can be resolved by examining which of the three layers was challenged, because each layer is independently verifiable against its component hash.

Any dispute can be resolved by examining which layer was altered.

10.2 BIP-341 Taproot Tweak

The Taproot commitment uses BIP-341's tagged hash construction:

```
tweak = SHA256("TapTweak" || SHA256("TapTweak") || P_int || C_stack) P' =
P_int + tweak · G
```

(Note: The current implementation simulates secp256k1 operations. A production deployment requires a native secp256k1 library. See Section 18.1.)

The commitment C_stack is embedded in the Taproot output key P' without appearing in any script or visible on-chain data. Only a party who knows P_int and C_stack can verify the relationship.

10.3 Satoshi Ordinal Binding

Rather than anchoring the stack commitment to a Bitcoin UTXO as a whole, the URIB binds C_stack to a specific, individually identifiable satoshi at **ordinal index 0** within the output UTXO, using the Ordinals protocol's convention for assigning persistent identity to individual satoshis within a transaction output. This granularity has a consequential property: the commitment travels with that specific satoshi through any subsequent transaction that spends the UTXO. As the satoshi moves through the Bitcoin transaction graph — spent, re-spent, and consolidated across any number of future transactions — the binding record moves with it, maintaining continuous provenance without requiring any additional on-chain write. Any party who can identify the committed satoshi by its ordinal index and UTXO reference can retrieve the canonical document hash, semantic graph hash, truth anchor, and stack commitment from the binding record, and independently verify the full settlement — including every pipeline stage and every rail state — without access to Jasper's internal state or any off-chain infrastructure beyond the Bitcoin blockchain itself.

---

11. Cross-Rail Mapping (BTC / XRP / ISO 20022 / CBDC)

11.1 The Principle of Value Equivalence

The URIB enforces a strict invariant: the value must be identical across all rails. For a settlement of USD 1,000:

For a settlement of USD 1,000, the value equivalence requirement applies across all four rails without tolerance for rounding or approximation. The ISO 20022 pacs.008 message instructs settlement of exactly USD 1,000 in the traditional correspondent

banking network, denominated in the instructed currency as supplied in the transaction document. The Bitcoin Taproot transaction carries the satoshi equivalent of USD 1,000 at the exchange rate fixed at Stage 6 — the rate is locked at the moment the rail states are computed and does not float between pipeline stages. The XRP payment instruction transfers the equivalent value denominated in XRP drops, converted at the same fixed rate. The wholesale CBDC proof records USD 1,000 in the native denomination of the CBDC rail, expressed in its protocol-specific representation. The value is not approximate: the cross-rail invariant `val(S_BTC) = val(S_XRP) = val(S_ISO) = val(S_CBDC)` must hold exactly, and any deviation — whether from rounding, conversion error, or a malformed rate fixture — causes the pipeline to fail with a cross-rail invariant violation before any settlement message is emitted to any rail.

If any rail's value does not match, the pipeline fails with a cross-rail invariant violation.

11.2 Rail-Specific State Representation

Each rail receives a state object tailored to its protocol:

ISO 20022:

```
{   "message_type": "pacs.008.001.10",   "end_to_end_id": "MSG<id>",
"debtor_agent": "BIC_DEBTOR",   "creditor_agent": "BIC_CREDITOR",
"instructed_amount": { "amount": 1000, "currency": "USD" },
"settlement_method": "CLRG",   "urib_hash": "<C_stack>" }
```

Bitcoin:

```
{   "protocol": "taproot_bip341",   "satoshi_value": 100000000000,
"commitment": "<C_stack>" }
```

XRP:

```
{   "transaction_type": "Payment",   "amount_drops": "1000000000",
"destination_tag": 2718281828,   "memo": "<C_stack>" }
```

CBDC:

```
{   "cbdc_type": "wholesale",   "amount": 1000,   "currency": "USD",
"proof_hash": "<C_stack>" }
```

11.3 Identity Mapping

Each of the four settlement rails uses a structurally different identifier format for transacting parties, and the URIB applies the **identity mapping function** Ψ_r(DID) at Stage 6 to translate the user's decentralised identifier into the format required by

each rail. ISO 20022 requires an IBAN-format account identifier in the form `IBAN_<hash>`, conforming to the addressing conventions of the correspondent banking network. Bitcoin requires a bech32m Taproot address in the `bc1p<hash>` format specified by BIP-341, which encodes the Taproot output key as a native SegWit version 1 address. The XRP Ledger requires an r-address in the `r<hash>` format native to the XRPL account model. The wholesale CBDC rail requires a CBDC-native participant identifier in the `CBDC_<hash>` format. In each case, the identifier is derived by hashing the DID with a rail-specific derivation path rather than deriving rail identifiers directly from one another, ensuring that a party who learns one rail identity cannot compute any other rail identity from it.

Identifiers are derived via hashing rather than directly from each other, preventing cross-rail linkability.

11.4 Bridge Commitment

After all rail states are computed, a bridge commitment C_bridge binds them together:

```
C_bridge = H(canonical(sorted(rail_states)))
```

After all four rail-specific state objects have been computed at Stage 6, the URIB applies a **canonical sort** to the array of rail states — ordering them deterministically by rail identifier in a fixed alphabetical sequence (BTC, CBDC, ISO, XRP) — and then serialises the sorted array to a canonical byte representation with no optional formatting. The serialised array is hashed with SHA-256 to produce C_bridge. Because the input to the hash function is fully determined by the content of the four rail states and the canonical sort order, C_bridge is reproducible by any party who holds copies of the four rail state objects — no access to Jasper's internal state is required to verify it. Any modification to any rail state — any change to a settlement amount, a rail address, a denomination, or a commitment value embedded within a state object — alters the serialised input and produces a different hash, invalidating C_bridge. The bridge commitment is therefore proof not merely that each individual rail state was computed, but that all four rail states were computed together, from the same settlement event, using the same stack commitment as their binding proof, and are mutually consistent with one another and with the cross-rail value equivalence invariant.

---

12. Model Routing & LLM Selection

12.1 Overview

Jasper routes LLM calls across multiple model families. The model router selects the optimal model for each task based on user preference and task requirements.

12.2 Available Models

Model ID	Family	Strengths
`gpt_5_mini`	OpenAI GPT-5	Fast, efficient, general-purpose
`gpt_5_4`	OpenAI GPT-5	Balanced performance
`gpt_5_5`	OpenAI GPT-5	High-quality reasoning
`gemini_3_flash`	Google Gemini 3	Fast, multimodal
`gemini_3_1_pro`	Google Gemini 3	Advanced reasoning, multimodal
`claude_sonnet_4_6`	Anthropic Claude	Nuanced reasoning, writing
`claude_opus_4_6`	Anthropic Claude	Deep reasoning, analysis
`claude_opus_4_7`	Anthropic Claude	Latest Opus tier
`claude_opus_4_8`	Anthropic Claude	Cutting-edge Opus
`claude_sonnet_5`	Anthropic Claude	Next-generation Sonnet

12.3 Routing Logic

14. **User preference** — if the user has selected a preferred model, that model is used unless the task clearly demands otherwise.
15. **Automatic mode** — if preferred_model is automatic or unset, Jasper selects based on:
    - Task complexity (simple → mini/flash; complex → opus/pro)

    - Task type (code → strong code model; creative writing → Claude; multimodal → Gemini)
    - Context length requirements
    - Latency sensitivity

12.4 Free LLM Router

A secondary router provides access to free-tier models, useful for:

- Low-stakes tasks where cost matters
- Batch processing
- Fallback when primary models are unavailable

---

13. Connected Apps & Universal App Connector

13.1 Overview

The **ConnectedApp** system is an abstraction layer that sits between the Jasper orchestration core and the fragmented external application ecosystem. It provides a uniform interface for reading data, invoking APIs, and coordinating multi-step workflows across heterogeneous third-party systems — insulating the orchestration core from the implementation details of any individual application and presenting the full breadth of connected software as a single, queryable surface.

From the orchestration core's perspective, all connected applications speak a common language. Each application declares its available entities, exposes its query and write interfaces, and reports its connection status through a standardised schema. The orchestration layer does not need to know the internal architecture of any individual application; it routes requests to the correct application based on entity type alone, making the underlying application topology transparent to both the agent and the user.

13.2 Connected App Schema

```
{   "app_id": "base44_app_123",   "app_name": "CRM System",   "api_key":
"<secure_key>",   "description": "Customer relationship management",
"is_active": true,   "available_entities": ["Customer", "Deal", "Contact",
"Lead"] }
```

13.3 Entity Discovery

Entity discovery is the mechanism by which the orchestration layer builds its internal map of the connected application ecosystem. At connection time, each app dynamically declares the entity types it exposes — for example, `Customer`, `Deal`, `Invoice`, `Project`, or `Payment` — and Jasper reads this declaration to register which data types are accessible through which application. The entity map is updated whenever a connection is activated, deactivated, or modified, keeping the orchestration layer's view of the application ecosystem current.

The operational consequence of entity discovery is that the user does not need to specify which application holds a given piece of data. When the orchestration core needs a `Deal` record, it consults the entity map, routes the query to the application that declared `Deal` as an available entity type, and returns the result — transparently, without requiring the user to manage the underlying data topology. This is what makes cross-application workflows possible: a single request can pull a customer record from the CRM, retrieve payment history from a financial application, and update a project tracker, because the orchestration layer knows, from the entity map alone, exactly where each data type lives.

13.4 Connection Lifecycle

Apps can be activated and deactivated without losing their configuration. This allows:

- Temporary disconnection during maintenance
- Cost management (deactivating infrequently used connections)
- Access control (deactivating when a user loses authorisation)

---

14. Asset Records & Tokenized Physical Assets

14.1 Overview

The AssetRecord system manages physical assets — primarily precious metals — with digital provenance, GPS tracking, vault storage, and Bitcoin anchoring via the Satoshi Covenant.

14.2 Asset Schema


```
{   "asset_id": "TX-GOLD-Bar-5092",   "asset_type": "gold",   "weight": 400,
"purity": 99.99,   "vault_location": "Zurich Vault 3, Bay 7",   "owner_did":
"did:jasper:user:abc123",   "gps_device_id": "GPS-DEV-7891",
"last_known_location": {     "latitude": 47.3769,     "longitude": 8.5417,
"speed": 0,     "heading": 0,     "timestamp": 1789234567   },
"satoshi_anchor": "<bitcoin_tx_id>",   "file_url":
"https://storage.base44.com/asset/TX-GOLD-Bar-5092.pdf",   "is_encrypted":
false,   "description": "400 oz .9999 fine gold bar, serial #5092" }
```


14.3 Asset Types

Type	Description
`gold`	Gold bars, coins, and related instruments
`silver`	Silver assets
`platinum`	Platinum group metals
`other`	Other physical assets (diamonds, fine art, etc.)

14.4 Satoshi Covenant Anchoring

Each asset can be linked to a Bitcoin transaction via the `satoshi_anchor` field — a Bitcoin transaction ID that links the physical asset to a specific satoshi on the Bitcoin blockchain. This creates a digital-to-physical bridge: the ownership and provenance of the physical asset is anchored to an immutable on-chain record.

14.5 GPS Tracking

Assets with GPS devices report their location periodically. The `last_known_location` record includes coordinates, speed, heading, and timestamp — enabling movement monitoring and geofencing via proactive tasks.

14.6 Document Storage

Each asset can carry an associated PDF document (certificate of authenticity, assay report, etc.) stored via Base44 storage, with the URL recorded in the asset record.

14.7 Encryption Flag

The `is_encrypted` flag indicates the asset's sensitive details are protected. In a production implementation, this corresponds to actual field-level encryption.

---

15. Security, Identity, and Post-Quantum Commitments

15.1 Decentralized Identifiers (DIDs)

Jasper uses DIDs as the primary identity format: `did:jasper:user:<id>`, `did:jasper:agent:<name>`, `did:jasper:sys:urib`. DIDs provide:

- **Self-sovereign identity** — no dependence on a central identity provider
- **Cross-system portability** — the same DID operates across all rails
- **Privacy** — DIDs do not directly reveal personal information

15.2 Post-Quantum Commitment Stamps

In accordance with JIP-QRM-URIB-01 §6, each URIB settlement is stamped with a post-quantum digital signature using ML-DSA-65 (Module-Lattice Digital Signature Algorithm, FIPS 204).

The PQ commitment process:

16. Compute the commitment root: `commitment_root = H(C_stack ‖ C_bridge)`
17. Retrieve the active PQ signing key from the KeyRegistry (`surface: urib`, `key_type: MLDSA65_SIGNING`, `status: active`)
18. Sign the commitment root with the PQ key
19. Attach the signature, key ID, pair ID, and signer DID to the settlement

```
{   "commitment_id": "URIB-CMT-<base36_timestamp>",   "commitment_version": 3,   "crypto_profile": "PQ_NATIVE",   "commitment_root": "<hash>", "commitment_signature_pq": "<base64_signature>",   "signer_did": "did:jasper:sys:urib",   "signer_key_id": "<key_id>",   "signed_at": "2026-07-19T23:44:53Z" }
```

15.3 Key Management

PQ signing keys are managed through a KeyRegistry entity with:

- `surface` — the system the key serves (e.g., `urib`)
- `key_type` — the cryptographic algorithm (e.g., `MLDSA65_SIGNING`)
- `status` — `active` or `rotated`
- `private_material` — the key material (base64-encoded)
- `pair_id` — links signing and verification key pairs
- `agent_name` — the agent that owns the key

15.4 Authentication

Every URIB operation requires an authenticated user. The pipeline invokes `base44.auth.me()` to verify identity before processing. Unauthenticated requests receive a 401 response; no processing occurs.

---

16. Scope Guardrails & Role Boundaries

16.1 Purpose-Built Agent

Jasper is a **purpose-built agent**: its operational role is defined at deployment time through an agent identity document that specifies the domains it operates in, the systems it has access to, and the classes of request it is designed to handle. This identity document constitutes the agent's operational contract, and every subsequent request is evaluated against it. Requests that advance the agent's defined role through its connected tools, memory, knowledge graph, and domain logic are fulfilled without qualification. Requests that fall outside it are declined, redirected, or interpreted in the most role-relevant available reading.

The architectural argument for this constraint is that scope is a capability, not a limitation. A general-purpose assistant that attempts to respond to everything produces responses that are inevitably ungrounded in some domains — plausible in register but unsupported by live data, connected tooling, or verifiable context. A purpose-built agent that operates exclusively within its defined systems produces responses that are contextually grounded, tooling-backed, and fully auditable, because every action it takes can be traced back to a system it directly controls. Scope discipline is what makes the agent's responses trustworthy rather than merely fluent.

16.2 Scope Rules

A request is in scope if it advances the agent's defined operational role using the application's connected tools, memory, knowledge graph, or domain-specific logic. In-scope requests are fulfilled without qualification — the agent is designed to handle them with full access to its cognitive and operational systems, and it does so without hedging, deferral, or partial engagement. The operational contract is not a preference to be weighed against user intent; it is the frame within which the agent's full capability is deployed.

When a request mixes in-scope and out-of-scope elements, the agent addresses the in-scope portion fully and declines the remainder with a brief, non-judgmental redirect. Wholly out-of-scope requests — general coding assistance, trivia, or creative writing unrelated to the user's work context — are declined without elaboration. Ambiguous requests are interpreted in the most role-relevant available reading; when the ambiguity cannot be resolved by interpretation, the agent asks a single short clarifying question rather than guessing or generating an ungrounded response.

Attempts to override, probe, or bypass the scope boundary are disregarded. The boundary is not a policy preference that can be negotiated in the course of a conversation — it is an architectural constraint enforced at the orchestration layer, applied consistently regardless of how a request is framed or what justification accompanies it.

16.3 Why Scope Matters

An unscoped agent is an unreliable agent. Maintaining clear boundaries allows Jasper to:

- Remain focused on capabilities it can execute reliably
- Avoid generating plausible-sounding but ungrounded responses
- Provide a predictable, trustworthy operational experience
- Keep the user's data and context within the systems designed to handle them

---

17. Performance Characteristics & Operational State

17.1 Stateless vs. Stateful Operations

- **Memory recall:** Query-based, O(n) with index support

- **Knowledge graph traversal:** Query-based, O(edges per node) for local traversal
- **URIB orchestration:** Stateless pipeline, O(1) per stage, with one audit log write
- **Proactive monitoring:** Polling-based, O(active tasks) per check cycle

17.2 Current System State (as of publication)

Subsystem	Status	Notes
MemoryBank	Operational	Read/write available
KnowledgeGraph	Operational	Node and connection CRUD available
EmotionalContext	Operational	Sentiment and tone detection available
ProactiveTask	Operational	No active tasks currently configured
URIB	Operational	Full 7-stage pipeline available
ConnectedApps	Operational	No apps currently connected
AssetRecords	Operational	No assets currently registered

17.3 Failure Modes & Handling

Scenario	Behaviour
Memory recall failure	Degrades gracefully; proceeds without recalled context
Knowledge graph query failure	Returns empty results; agent continues
URIB semantic invariant violation	Pipeline aborts at Stage 2; returns 422
Cross-rail value mismatch	Pipeline aborts at Stage 6; returns 422
PQ key unavailable	Commitment stamp skipped;

Scenario	Behaviour
	settlement proceeds without PQ stamp
Audit log write failure	Non-blocking; settlement completes
Unauthenticated request	401 returned; no processing

---

18. Future Directions

18.1 Real Cryptography

The current implementation simulates secp256k1 operations (Taproot tweak computation) and uses HMAC as a stand-in for ML-DSA-65 signatures. Production deployment requires:

- Native secp256k1 library integration
- A conformant ML-DSA-65 implementation or integration with a certified post-quantum cryptography library
- Hardware security module (HSM) storage for signing keys

18.2 Live Rail Integration

The URIB currently produces settlement messages but does not broadcast them to live rails. Future work includes:

- **ISO 20022:** SWIFT network integration or direct bank API connectivity
- **Bitcoin:** Bitcoin node integration for Taproot transaction broadcast
- **XRP:** XRPL node integration
- **CBDC:** Central bank API integration (where available)

18.3 Advanced Knowledge Graph

- Graph algorithms (shortest path, centrality, community detection)

- Automated node creation from conversation analysis
- Cross-user knowledge sharing with appropriate consent controls

18.4 Enhanced Proactive Monitoring

- ML-based anomaly detection
- Natural language trigger conditions ("alert me when something unusual occurs in the portfolio")
- Cross-task coordination (multiple tasks sharing monitoring infrastructure)

18.5 Multi-Agent Coordination

- Agent-to-agent communication protocol
- Specialised sub-agents (finance, research, scheduling) coordinated by a master orchestrator
- Shared memory and knowledge graph across agents in a coordinated pool

---

19. Conclusion

Jasper represents a complete agentic orchestration architecture — not a chatbot with plugins, but an integrated platform in which memory, knowledge, emotional calibration, proactivity, financial settlement, and cryptographic accountability are woven into a single coherent agent experience.

The key architectural innovations are:

20. **ThreadZero** — a hash-chained truth chain that makes every action auditable and tamper-evident
21. **Stack Commitment** — a single cryptographic binding of document, semantics, and processing history
22. **Cross-Rail Value Equivalence** — enforced consistency across Bitcoin, XRP, ISO 20022, and CBDC simultaneously

23. **Two-Tier Memory** — lightweight session memory operating alongside structured, typed, importance-weighted persistent memory
24. **Emotional Calibration** — adaptive tone and response style grounded in real-time state detection
25. **Scope Discipline** — a purpose-built agent with a defined operational role, maintained without exception

The result is an agent that does not merely respond to queries. It remembers, reasons, monitors, routes, settles, and proves — across every session, every subsystem, and every rail it is connected to.

---

20. Appendices

Appendix A: URIB Orchestration Request Schema


```
{   "action": "orchestrate",   "raw_doc": {     "debtor": "Alice Corp",
"creditor": "Bob Ltd",     "amount": 50000,     "currency": "USD",
"message_id": "MSG-2026-0001"   },   "actor": "did:jasper:user:abc123",
"did": "did:jasper:user:abc123",   "rails": ["ISO", "BTC", "XRP", "CBDC"],
"utxo_ref": "utxo:genesis:0",   "p_int": "jasper_internal_key_v1" }
```


Appendix B: URIB Orchestration Response Schema


```
{   "success": true,   "pipeline": "URIB_ISO20022",   "h_doc": "<sha256>",
"h_sem": "<sha256>",   "semantic_graph": { "nodes": 4, "edges": 3, "meta": {}
},   "thread_anchor": "<T*>",   "event_count": 6,   "c_stack": "<sha256>",
"taproot_key": "taproot_<hex>",   "satoshi_bindings": [{ "ordinal_index": 0,
"utxo_ref": "..." }],   "rail_states": { "ISO": {}, "BTC": {}, "XRP": {},
"CBDC": {} },   "rail_identities": { "ISO": {}, "BTC": {}, "XRP": {}, "CBDC":
{} },   "cross_rail_invariants_pass": true,   "c_bridge": "<sha256>",
"iso20022_messages": [],   "btc_tx": {},   "xrp_tx": {},   "commitment": {
"commitment_id": "URIB-CMT-<id>",     "crypto_profile": "PQ_NATIVE",
"commitment_root": "<hash>",     "commitment_signature_pq": "<base64>"   },
"audit": {     "audit_type": "ThreadZero",     "truth_anchor": "<T*>",
"event_count": 6,     "stack_commitment": "<C_stack>",
"bridge_commitment": "<C_bridge>",     "events": []   } }
```


Appendix C: Memory Type Reference

Type	Use Case	Default Importance
`fact`	Objective information	5
`preference`	User preferences	5
`goal`	Objectives	7
`relationship`	People connections	6
`project`	Ongoing work	7
`conversation_highlight`	Key moments	5

Appendix D: Glossary

Term	Definition
URIB	Universal Rail Integration Bridge — the 7-stage settlement pipeline
ThreadZero	The hash-chained audit trail system
Stack Commitment (C_stack)	Hash binding document, semantic, and truth-chain layers
Bridge Commitment (C_bridge)	Hash binding all rail states together
Taproot	Bitcoin BIP-341 protocol for commitment embedding
Satoshi Covenant	Linking physical assets to specific satoshis via Bitcoin transactions
DID	Decentralized Identifier — self-sovereign identity format
PQ_NATIVE	Post-quantum native commitment profile using ML-DSA-65
pacs.008	ISO 20022 message type for credit transfers
Cross-Rail Equivalence	The invariant that value must be identical across all settlement rails
ML-DSA-65	Module-Lattice Digital Signature Algorithm, Level 3 (FIPS 204)
BIP-341	Bitcoin Improvement Proposal defining the Taproot output type